# Day 11 — React Components, State & Forms

Beginner-friendly notes and a complete Student Registration Form project

## Learning objectives

• Understand functional components and props.

• Manage changing data with the useState hook.

• Handle input changes and form submission.

• Build controlled form fields and basic validation.

• Use conditional rendering to show submitted student details.

## 1. Functional components

A functional component is a JavaScript function that returns JSX (the UI). Components help split a page into small, reusable parts.

```
function Welcome() {
  return <h2>Welcome to React!</h2>;
}

export default Welcome;
```

## 2. Props

Props are values passed from a parent component to a child component. They let a component display data supplied by another component.

```
function Greeting({ name }) {
  return <p>Hello, {name}!</p>;
}

// Use it inside another component:
<Greeting name="Vaibhav" />
```

## 3. State and useState

State is data a component remembers. When state changes using its setter function, React renders the updated UI.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

Syntax: const [value, setValue] = useState(initialValue);

## 4. Event handling

Events let your app respond to user actions. In JSX, use camelCase event names such as onClick, onChange, and onSubmit.

```
<button onClick={() => alert("Clicked!")}>
  Click me
</button>
```

## 5. Controlled components

A controlled input gets its value from React state. onChange updates that state whenever the user types.

```
const [name, setName] = useState("");
```

```
<input
  value={name}
  onChange={(event) => setName(event.target.value)}
/>
```

## 6. Form handling and validation

Use onSubmit to handle form submission. event.preventDefault() prevents a full browser page reload. Validate the input before displaying success.

```
function handleSubmit(event) {
  event.preventDefault();

  if (!name.trim()) {
    setError("Name is required");
    return;
  }

  setError("");
  // Continue with valid data
}
```

## 7. Conditional rendering

Conditional rendering shows UI only when a condition is true. The && operator is a common pattern.

```
{student && (
  <p>Registered: {student.name}</p>
)}
```

# Practical: Student Registration Form

## Project features

• Fields: student name, email, and course.

• Validation: name is required and at least 3 characters; email must have a basic valid format; course must be selected.

• State: inputs, validation errors, and submitted student record are stored with useState.

• Props: StudentDetails receives the submitted student object from App.

• Clear button resets the form and displayed result.

## Setup commands

```
npm create vite@latest student-registration -- --template react
cd student-registration
npm install
npm run dev
```

If your system has a no-space-left error, free disk space first or create the project on a drive with enough free space.

## Project file structure

```
student-registration/
  index.html
  package.json
  src/
    main.jsx
    App.jsx
    App.css
```

## Complete project code

The downloadable project ZIP contains the complete code in separate files. Extract it, open that folder in VS Code, then run npm install and npm run dev.

## How the project works

**useState:** Stores name, email, course, validation errors, and the submitted student record.

**onChange:** Updates the state when the user edits a field.

**onSubmit:** Runs validation and prevents the default page reload.

**Validation:** Shows an error for missing/invalid data and only accepts valid entries.

**Props:** Passes the student object to StudentDetails.

**Conditional rendering:** StudentDetails displays nothing until a valid form submission exists.

## Testing checklist

■ Submit with all fields empty: validation errors should appear.

■ Enter a name with fewer than 3 characters: name error should appear.

■ Enter an invalid email such as abc: email error should appear.

■ Leave the course unselected: course error should appear.

■ Enter valid details and submit: submitted details should appear.

■ Click Clear form: fields, errors, and result should reset.

## Important note

This is a frontend-only learning project. Submitted details exist in React state and are not stored permanently or sent to a backend/database.